

## 46-927, Spring 2021: Homework #6

Due 1:30PM, Wednesday, March 10, 2021.

**For this homework, we're back to Python with PyTorch.** Problem 1 should be the only part with substantial ML work; problems 2 and 3 are mainly about exploring how PyTorch works and trying some example problems.

You will find starter code for this homework in `hw6_starter.ipynb`. Most of the detailed instructions are in comments in that code, rather than this PDF.

1. *Backpropagation for linear regression.* In this problem, you will be implementing linear regression using the ideas we've learned for fitting neural networks: backpropagation and gradient descent.

For this problem, you will be finding  $\beta$  to optimize:

$$\sum_{i=1}^n (y_i - x_i^T \beta)^2$$

You will think of this as the composition of two functions:

$$\begin{aligned}\mathcal{L}(\hat{y}) &= \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{2} \|y - \hat{y}\|_2^2 \\ \hat{y} &= g(X) = X\beta\end{aligned}$$

*Note:* Section 6.5 of *Deep Learning* (<https://www.deeplearningbook.org/>) covers backpropagation in detail, so you can see a mathematical derivation of it. You can also start with Problem 2 to explore PyTorch before returning to Problem 1 as we cover backpropagation.

- (a) In the starter code, fill in the functions marked **TODO** in the starter code for the **Loss** and **Linear** classes. The basic idea is that **forward** should compute the desired output for each class (evaluation of the linear function or evaluation of the loss), and also store any important objects needed for the gradient. The **backward** function will return the gradient of the function's output with respect to its input, using those stored values from the last evaluation.

There are different conventions that you could use for your gradient shapes, but it's important that you conform to one so that all your multiplications work. In this code, it is assumed that every gradient is shape

$$(\text{number of parameters/inputs}) \times (\text{number of outputs}).$$

Assert statements have been included to help you verify this and aid in debugging.

Check your code in the block marked “Check 1a,” which will evaluate each of these functions and their returned gradients. Those responses are part of your answer to this question.

- (b) Set up the complete evaluation of one cycle of evaluations: Forming a prediction, evaluating the loss, and producing an overall gradient for the derivative of loss by beta. See comments in the provided code for details.
- (c) Finally, implement gradient descent! See comments in the provided code for details.
- (d) Look at the output from your optimizer! Is your solution close to the desired  $\beta^* = (3, 1, 4)^T$ ?

Plot the loss as a function of iteration number. Does it seem to smoothly decrease toward zero?

- (e) *Stochastic gradient descent.* In the last optimizer, you looked at every data point at every iteration. This is feasible on this small problem, but would not work well for large neural networks on large data sets. Instead, some version of stochastic gradient descent is used, where the gradient is evaluated only for a subset of data points at any one time.

Write another version of your solver that uses stochastic gradient descent. We’ll be using the simplest version: each iteration looks at the performance and gradient of your loss for only *one* data point! See detailed instructions in the notebook file.

After 1000 iterations, how does your beta estimate look compared to the least-squares estimate?

Plot both measures of losses as a function of iteration. These should be decreasing more slowly and noisily toward zero than your original solver.

2. *Exploring PyTorch.* PyTorch is a useful library for fitting neural networks. It includes many of the components necessary for implementing neural networks, as well as autodifferentiation through backpropagation, and tools for things like data set handling and sampling.

For this problem, you will replicate problem 1 (c) using built in PyTorch functions. See the starter code for problem 2 for details of what you should be doing. It pretty much walks you through it line by line. Try to appreciate that (1) all the functions

you need have their backward steps coded for you, and (2) by making things into `Variables`, Torch tracks all of the operations and decides what to multiply together to get your final gradient.

Along with your code, produce a plot of the losses during training, and the final beta vector estimate.

(There is very little to actually do in this problem. The ease with which this can be implemented is pretty cool.)